FACULTAD DE CIENCIAS

MATERIA: ICC

ESCUELA: UNAM

Número de cuenta **324181883**
Nombre del alumnno: **Melissa Prado Hernandez**
Número de practica: **Prcatica07**
Fecha de entrega: **9 de Noviembre del 2026**

eginfigure  .

1 Parte 1: Ánalisis Gramatical y Modelado

La primera parte de la práctica me pedia realizar un diagrama de clase con sus respectivos atributos, métodos y el nombre de la clase.
Texto del problema: "Una empresa de entrega de paquetes lleva varios años operando en la ciudad con gran éxito y reconocimiento por su puntualidad. Recientemente, la demanda de envíos ha crecido de forma considerable, en especial durante temporadas festivas y promociones de fin de año, por lo que el personal administrativo ha tenido dificultades para dar seguimiento manual a las órdenes. Para atender esta situación, la dirección ha decidido solicitar el desarrollo de un nuevo sistema que gestione sus envíos postales de forma confiable. Cada paquete que ingresa al almacén se identifica mediante un código de rastreo alfanumérico único, y registra su peso físico en kilogramos, sus medidas espaciales (alto, ancho y largo en centímetros) y el valor monetario declarado de su contenido para efectos del seguro aduanal. El sistema debe ser capaz de calcular el peso volumétrico del paquete, determinar el peso cobrable final y cotizar el importe total del envío aplicando una tarifa por rangos de peso."
Lo primero que debemos identificar es la clase para poder sacar los atributos y métodos respectivos. En mi caso para encontrar los métodos buscare los verbos, acciones que la clase debe de hacer o tener. Ahora para el caso de los atributos lo primero es buscar que es lo que tiene esa clase, que contiene, una vez ya teniendo en cuenta lo siguiente, procedi separando cada cosa en su respectivo lugar.

La Clase

Al leer el texto, logré identificar que la clase era "Envío", ya que todo giraba alrededor de esto, otro punto importante para identificarla fácilmente fue que explícitamente la describen como el problema a resolver. Una vez terminado esto, ubiqué lo que era la "paja" del problema: datos innecesarios como los años que llevaba operando la empresa, si es reconocida o no, y cuáles eran sus temporadas de mayor demanda durante el año.

Atributos

En la parte de los atributos, una estrategia para mi es preguntarme que datos tiene mi clase y me pregunté: ¿Qué datos tiene o contiene un paquete?. A partir de esto, identifiqué en el texto los siguientes elementos esenciales:

1. Código de rastreo el alfanumérico único, esto es para identificar el paquete de cada individuo.
2. Peso físico en Kilogramos, esto para saber su peso.
3. Medidas especilaes (alto, ancho y largo).
4. El Valor Monetario, esto es requerido para efectos del seguro aduanal.

Métodos

En la parte de los métodos, igualmente me hago una pregunta ¿Qué acciones debe realizar?. Otra opción que tome en cuenta es buscar los verbos del texto como:

1. Calcular el Peso Volumétrico
2. Determinar el Importe total
3. Cotizar la tarifa de Precios

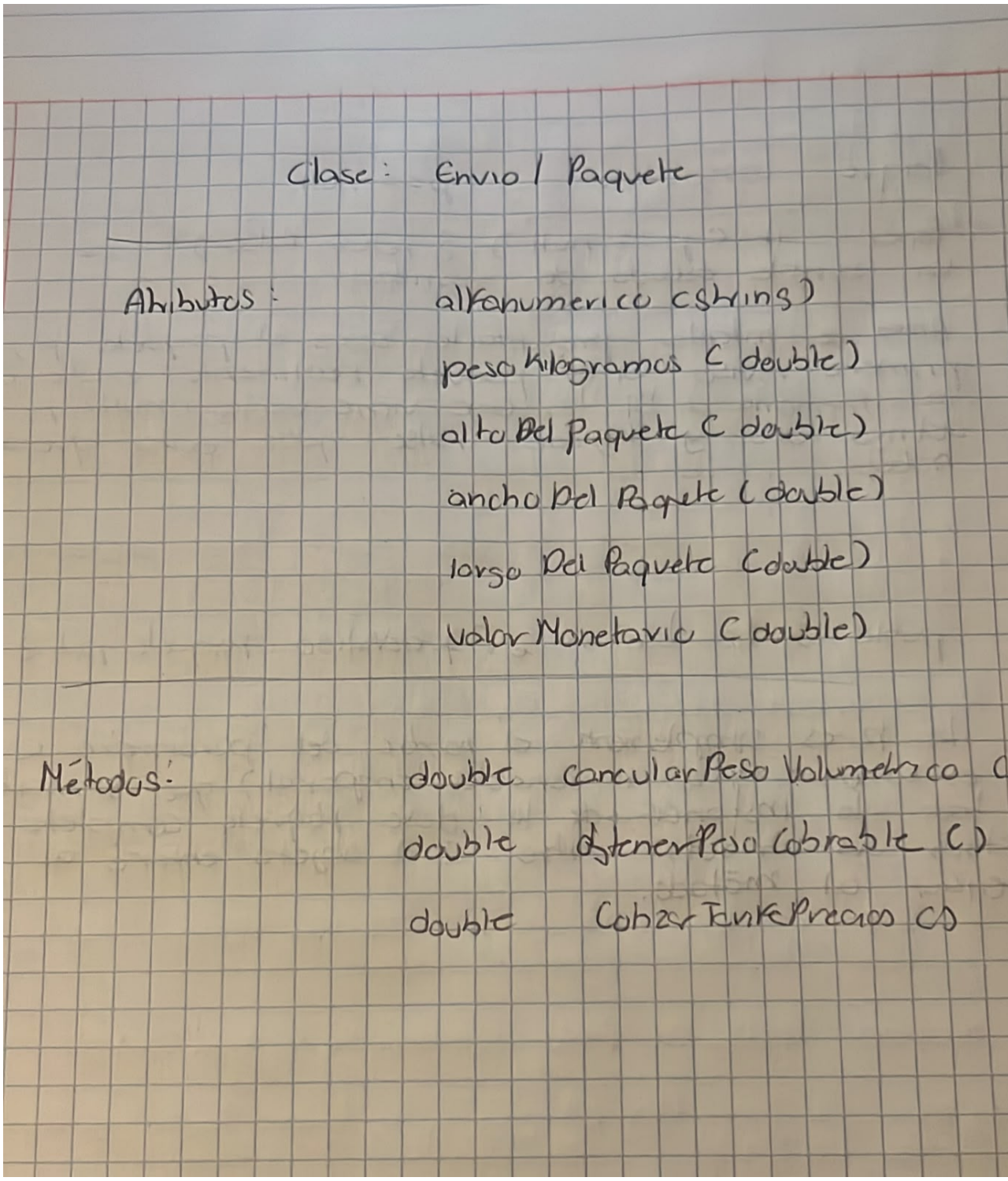


Figure 1: Tabla de mi Diagrama de la clase

Algo para aclarar es que en la parte de la clase tanto podria ser tanto el nombre de "Envio" o podria ser "Paquete", porque en cualquiera de los dos casos tienen los mismos atributos y métodos.

2 Parte 2: Implementación de la clase paquete

CLASE PAQUETE
Para la primera clase de "Paquete" para esta clase primero era definir los atributos que ya habia clasificado anteriormente en el diagrama. Dichos atributos quedaron estructurados de la siguiente forma:

1. private double pesoKilogramos;
2. private double altoDelPaquete;
3. private String alfanumerico;
4. private double anchoDelPaquete;
5. private double largoDelPaquete;
6. private double valorMonetario;

Una vez implementado el método de validación, continué con la elaboración del método cotizarEnvio, diseñado con la firma public static double cotizarEnvio(Paquete p) para retornar el costo total en formato decimal sin necesidad de instanciar la clase. Lo primero que realiza este método es invocar a esPaqueteValido(p); si la validación me regresa un resultado falso (esPaqueteValido(p) == false), detengo inmediatamente el cálculo retornando -1.0 como un código de error que indica que el envío no es cotizable. En caso de superar la validación, llamo al método del objeto p.obtenerPesoCobrable() para consultar y almacenar en la variable local peso la cantidad mayor entre el peso físico y el volumétrico. Posteriormente, mediante una estructura condicional if / else if / else, aplico la tarifa escalonada de la empresa: si el peso cobrable es menor o igual a 5.0 kg, se retorna la tarifa base fija de 120 pesos si se encuentra en el rango de 5.01 kg a 15.0 kg, devuelvo la cuota base más el cobro extra de 18 pesos por cada kilogramo excedente sobre los primeros 5 kg (120.00 + (18.00 * (peso - 5.0))); y finalmente, si el peso sobrepasa los 15.0 kg, cae en el bloque else, aplicando una tarifa de 200 pesos más 25 pesos por cada kilogramo excedente sobre los 15 kg (300.00 + (25.00 * (peso - 15.0))).Todos los atributos se declararon con el modificador private aplicando el principio de encapsulamiento visto en clase. Posteriormente a esta sección, procedí a implementar la sobrecarga de constructores que igualmente se vio en clase. Primero el constructor sin parámetros (predeterminado) y sucesivamente, agregué parámetros mientras se iba sumando más constructores hasta llegar a tener todos los parametros.

public Paquete (String alfanumerico, double pesoKilogramos, double altoDelPaquete, double largoDelPaquete, double anchoDelPaquete, double valorMonetario.) this.alfanumerico = alfanumerico; this.pesoKilogramos = pesoKilogramos; this.altoDelPaquete = altoDelPaquete; this.largoDelPaquete = largoDelPaquete; this.anchoDelPaquete = anchoDelPaquete; this.valorMonetario = valorMonetario;

Después de terminar con los constructores, empece con los métodos de conuslta, esto para que puedan leer los atributos de forma segura.

public String getAlfanumerico() return this.alfanumerico;

Este procedimiento lo realice para cada uno de los atributos.

Al finalizar los getters, continué con la implementación de los métodos de asignación (setters), encargados de modificar los valores de los atributos mediante validaciones y reglas. La lógica aplicada verificó que el valor fuera estrictamente mayor a 0 y en caso de cumplirse la condición, el valor se asigna correctamente al atributo. En caso contrario si el valor es igual o menor a 0, le aparece al usuario un mensaje en su pantalla que el tipo de dato que ingreso no es válido.

public void setAlfanumerico(String alfanumerico) this.alfanumerico = alfanumerico;
De igual forma, establecí la misma condición para cada uno de mis atributos

Finalmente, para concluir la clase Paquete implementé los métodos de cálculo requeridos en la práctica. El primero de ellos corresponde a la determinación del peso volumétrico a partir de la fórmula que me proporcionaron en la práctica.

public double calculaPesoVolumetrico() return (this.largoDelPaquete * this.anchoDelPaquete * this.altoDelPaquete) / 5000.0;

Una vez calculado el peso volumétrico, procedí a implementar el método para obtener el peso cobrable para ello, almacené primero el resultado de calculaPesoVolumetrico() en una variable que estuviera dentro de mi condición denominada pesoVolumetrico, comparándolo mediante una condicional contra el peso físico. Tal cual como fue descrito en la práctica.

public double obtenerPesoCobrable() double pesoVolumetrico = calculaPesoVolumetrico(); if (this.pesoKilogramos menor que pesoVolumetrico) return this.pesoKilogramos; else return pesoVolumetrico;

CLASE COTIZADORENVIOS

Para la segunda clase a diferencia de la clase Paquete, esta clase no contiene los atributos, ya que en la primera clase ya estan definidas. Por lo tanto creé

un nuevo constructor.

public CotizadorEnvio()

Posteriormente de crear el constructor, procedí a implementar los métodos estáticos exigidos por las reglas de negocio. El primer método desarrollado fue esPaqueteValido, encargado de validar que se cumpla con todo loo que se establecio en la anterior clase como que sus números de todos sean positivos al momento de ingresarlo.

Lo primero que implementé fue la condición if (p == null) como un escudo de seguridad fundamental en Java y esto me permite verificar si la referencia del objeto viene totalmente vacía en memoria para detener la función regresando false y evitar así que el programa colapse leer un objeto inexistente. Una vez asegurado que el paquete sí existe en memoria, utilicé los métodos de consulta (getters) como getPesoKilogramos(), getAltoDelPaquete(), getAnchoDelPaquete(), getLargoDelPaquete() y getValorMonetario() para consultar y evaluar de forma individual cada uno de sus datos. Implementé condicionales if para que evaluara las reglas de la empresa que ni el peso ni las demas mencionadas sean iguales o menores a cero, porque no pueden existir paquetes de 0 kg o 0 cm, y que el valor monetario no sea negativo. Si alguna de estas condiciones no se cumple, el método se detiene de inmediato devolviendo false para indicar que el paquete es inválido, pero si la ejecución logra superar todas estas revisiones sin activar ningún if, el método concluye retornando true y esto confirma que el paquete cuenta con datos válidos y está listo para ser cotizado en el sistema.

public static boolean esPaqueteValido(Paquete p) if (p == null) return false; if (p.getPesoKilogramos() menor o igual 0) return false; if (p.getAltoDelPaquete() menor o igual 0) return false; if (p.getAnchoDelPaquete() menor o igual 0) return false; if (p.getLargoDelPaquete() menor o igual 0) return false; if (p.getValorMonetario() menor 0) return false; return true;

Terminando con esto, para saber si el paquete era valido para ser cotizado, continué con la elaboración del método cotizarEnvio. Este método se encarga de determinar el costo final del envío aplicando la tarifa escalonada según el rango en el que se ubique el peso cobrable del paquete.

public static double cotizarEnvio(Paquete p) if (esPaqueteValido(p) == false) return -1.0;

Para mi siguiente condición, si la validación me regresa un resultado falso (esPaqueteValido(p) == false), automaticamente me dara -1.0 como un código de error que indica que mi paquete no es cotizable. En caso contario llamo al método del objeto p.obtenerPesoCobrable() para consultar y almacenar en la variable local peso la cantidad mayor entre el peso físico y el volumétrico. Posteriormente mediante condicionales aplico la tarifa escalonada de la empresa:

si el peso cobrable es menor o igual a 5.0 kg, se retorna la tarifa base fija de 120 pesos, si se encuentra en el rango de 5.01 kg a 15.0 kg, devuelvo la cuota base más el cobro extra de 18 pesos por cada kilogram y si excedente sobre los primeros 5 kg (120.00 + (18.00 * (peso - 5.0))). Finalmente, si el peso sobrepasa los 15.0 kg, cae en el bloque else, aplicando una tarifa base de 300 pesos más 25 pesos por cada kilogramo excedente sobre los 15 kg (300.00 + (25.00 * (peso - 15.0))).

double peso = p.obtenerPesoCobrable();
if (peso menor o igual 5.0) return 120.00; else if (peso menor o igual 15.0) return 120.00 + (18.00 * (peso - 5.0)); else return 300.00 + (25.00 * (peso - 15.0));

Finalmente para concluir con la clase CotizadorEnvio, desarrollé el método requiereSeguro, definido como public static boolean requiereSeguro(Paquete p) para evaluar si la protección es obligatoria. Al igual que en la cotización, el método inicia evaluando si el paquete no es válido para descartar de inmediato el seguro devolviendo false. Si el paquete es válido procedo a evaluar de forma secuencial las tres políticas de riesgo mediante las condicionales if. La primera condición utiliza el getter p.getValorMonetario() para verificar si el valor declarado del contenido supera el límite permitido de 3000 pesos . La segunda condición evalúa que el paquete cumpla con las condiciones de la estructura física utilizando los getters de las dimensiones combinados con el operador lógico OR (——), en caso que alguna de sus caracteristicas requeridas (alto, ancho o largo) excede de forma individual los 60.0 cm, la regla se activa. La tercera condición calcula la suma total del perímetro acumulado almacenando p.getAltoDelPaquete() + p.getAnchoDelPaquete() + p.getLargoDelPaquete() en la variable local sumaLados y comprueba si se pasa de los 150.0 cm combinados. Si cualquiera de estos tres if se cumple, el método se interrumpe retornando true, esto indicando que el paquete exige seguro obligatorio, y si el paquete no incumple ninguna de estas tres normas la función llega a su línea final devolviendo false para indicar que el seguro no es necesario.

if (p.getValorMonetario() menor que 3000.00) return true;
if (p.getAltoDelPaquete() mayor que 60.0 or p.getAnchoDelPaquete() mayor que 60.0 or p.getLargoDelPaquete() mayor que 60.0) return true;
double sumaLados = p.getAltoDelPaquete() + p.getAnchoDelPaquete() + p.getLargoDelPaquete(); if (sumaLados mayor que 150.0) return true;
return false;

Preguntas a responder

¿Por qué resulta conveniente que CotizadorEnvio tenga métodos estáticos en lugar de requerir crear instancias con new?
Porque solamente vamos hacer calculos, no vamos a guardar informacón dentro de ella.

¿Qué diferencia conceptual y práctica existe entre una variable de referencia con valor null y un objeto instanciado con su constructor por defecto?

Variable con null: No existe nada en memoria. La variable apunta a la nada, por lo que si intentas pedirle un dato, el programa truena inmediatamente con un error.

Objeto instanciado: Sí existe en memoria. Ocupa un lugar real y tiene atributos inicializados con valores por defecto (como 0 o null interno), por lo que el programa no se rompe al consultar sus datos; simplemente responde con ese valor inicial.

(La clase prueba de Paqueteria)
Este tipo de clase para prueba del codigo e interacción con el usuario.

Para finalizar toda la práctica, desarrollé la clase para verificar mi clase PruebaPaqueteria, la cual contiene el método public static void main(String[] args) para verificar que mi mis clases anteriores funcionen de la manera correcta. Empecé con los paquetes de prueba para hacer funcionar mis métodos (p1, p2, p3 y p4) con el operador new, asignándoles las caracteristicas que se piden en los métodos anteriores como el: pesos y valores específicos, destacando a p4 con datos inválidos (peso de -2.0 kg y alto de 0.0 cm) para evaluar la respuesta ante los errores que se presentan. Para no revolverme lo que hice fue poner condicionales para cada caso en especial de cada paquete en las que se llamo a la clase de CotizadorEnvio porque ahi tenia mis operaciones mediante la instrucción CotizadorEnvio.esPaqueteValido(p). Esta instrucción funciona consultando la clase estática CotizadorEnvio, la cual verifica y compara los atributos del objeto contra las reglas que se establecieron anteriormente en el negocio. Si el paquete cumple todas las condiciones, la función responde con un valor verdadero (true), lo que permite ingresar al bloque del if para calcular e imprimir en consola el código alfanumérico (p.getAlfanumerico()) más el costo (CotizadorEnvio.cotizarEnvio(p)) y la obligatoriedad del seguro (CotizadorEnvio.requiereSeguro(p)) mientras que si la evaluación falla, se desvía al bloque else notificando que el paquete no es válido.

System.out.println("Prueba de envios ");

/* Aqui creé los diferentes paquetes de prueba para verificar que el programa funciona, y creé uno es especifico que no cumplia los criterios **/ Paquete p1 = new Paquete("Paquete 1", 3.0, 20.0, 15.0, 10.0, 500.0); Paquete p2 = new Paquete("Paquete 2", 12.0, 40.0, 30.0, 25.0, 2500.0); Paquete p3 = new Paquete("Paquete 3", 18.0, 70.0, 50.0, 40.0, 4000.0); Paquete p4 = new Paquete("Paquete 4," -2.0, 0.0, 10.0, 10.0, -100.0); // — Probamos el Paquete 1 — System.out.println("— Probando Paquete 1 —"); if (CotizadorEnvio.esPaqueteValido(p1)) System.out.println("Codigo: " + p1.getAlfanumerico()); System.out.println("Costo total: pesos" + CotizadorEnvio.cotizarEnvio(p1)); System.out.println("Requiere seguro: " + CotizadorEnvio.requiereSeguro(p1)); else System.out.println("El Paquete 1 no es valido.");

Finalmente, para probar todos mis métodos tuve que crear otro paquete prueba para ppoder verificar cuando el paquete mide mas de 60 cm y me pide seguro obligatorio por exceder el límite espacial. Creé dos objetos adicionales: miPaquete1 con una altura de 65.0 cm para corroborar que aplique lo establecido y aunque su peso sea bajo, y miPaqueteNulo para verificar que la condición de seguridad evite el colapso, de que sea un paquete valido.

Paquete miPaquete1 = new Paquete("Mi paquete", 2.0, 65.0, 10.0, 10.0, 500.0);

// Creamos un paquete que vale null para probar que no se rompa el programa Paquete miPaqueteNulo = null;

// Probamos miPaquete1 if (CotizadorEnvio.esPaqueteValido(miPaquete1)) System.out.println("Mi Paquete 1 es valido."); System.out.println("¿Requiere seguro?: " + CotizadorEnvio.requiereSeguro(miPaquete1));

// Probamos miPaqueteNulo if (CotizadorEnvio.esPaqueteValido(miPaqueteNulo)) System.out.println("El paquete nulo es valido."); else System.out.println("El paquete nulo NO es valido (funciono la proteccion).");

Compilación final de todo mi programa

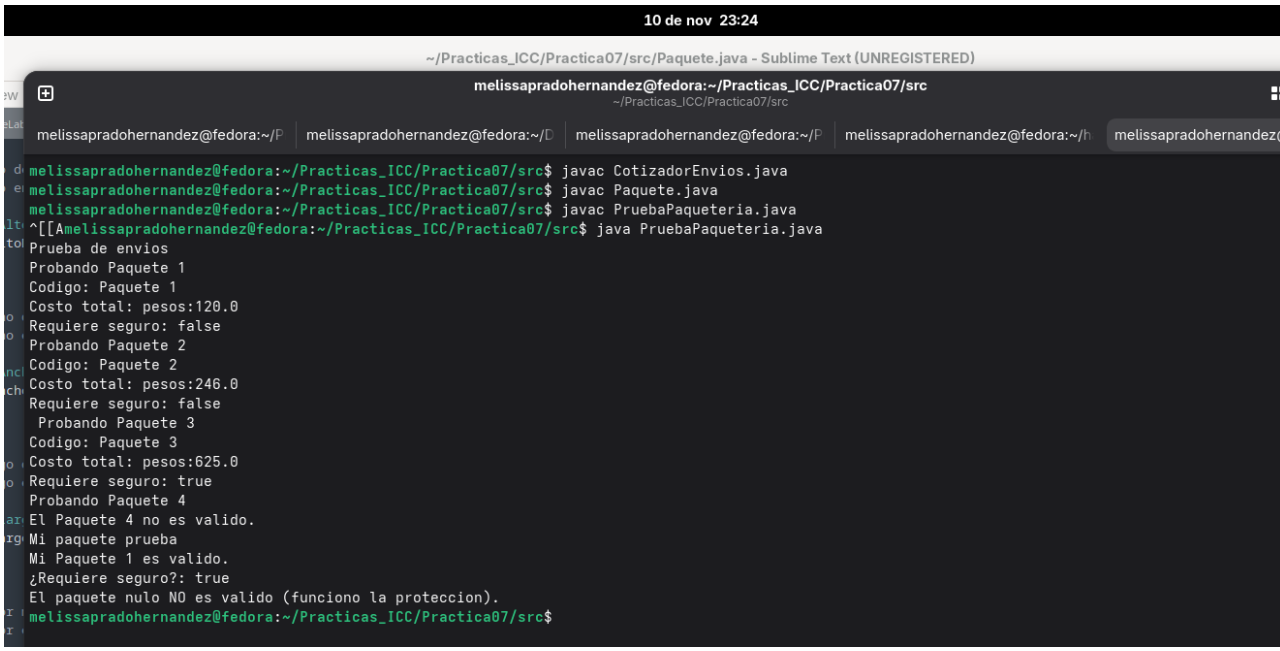


Figure 2: Compilació y ejecución de mi programa